

OpenMP Introduction

Ernesto Sanchez

Introduction

What is OpenMP?

Open Multi-Processing (OpenMP) is an Application Programming Interface (API) specification to support the shared-memory multiple-threaded form of parallel application development.

OpenMP is mainly composed of a set of compiler directives to mark parallel regions.

www.openmp.org – Talks, examples, forums, etc.

High-level API

- Preprocessor (compiler) directives (~ 80%)
- Library Calls (~ 19%)
- Environment Variables (~ 1%)

A Programmer's view of OpenMP

OpenMP is a portable, threaded, shared-memory programming *specification* with “light” syntax

- Exact behavior depends on OpenMP *implementation*!
- Requires compiler support (C or Fortran)
- Current version 6.0

OpenMP will:

- Allow a programmer to separate a program into *serial regions* and *parallel regions*, a kind of T concurrently-executing threads.
- Hide stack management
- Provide synchronization constructs.

OpenMP will not:

- Parallelize (or detect!) dependencies
- Guarantee speedup.

Parallel programming using OpenMP

1. Start with a *parallelizable* algorithm

Embarrassing parallelism is good, loop-level parallelism is necessary

2. Implement serially, *ignoring*:

- Data Races,
- Synchronization,
- Threading Syntax

3. Test and Debug

4. Annotate the code with parallelization and synchronization directives

- “magically” parallelize the program execution
- Expect for linear speedup

5. Test and Debug

First example – OpenMP

```
#include <stdio.h>

int main() {

    // Do this part in parallel

    printf( "Hola Mundo!\n" );

    return 0;
}
```

Parallelize using OpenMP

```
#include <stdio.h>
#include <omp.h>

int main() {

    omp_set_num_threads(4);

    // Do this part in parallel
    #pragma omp parallel
    {
        printf( "Hola Mundo!\n" );
    }

    return 0;
}
```

Parallelize using OpenMP

```
#include <stdio.h>
#include <omp.h>
```

```
int main() {
```

```
    omp_set_num_threads(8);
```

```
    // Do this part in parallel
```

```
    #pragma omp parallel
```

```
{
```

```
    printf( "Hola Mundo!\n" );
```

```
}
```

```
    return 0;
```

```
}
```

This directive can be replaced by an environmental variable:

> export OMP_NUM_THREADS=8

Compile an OpenMP program

Use the following command to compile the code:

```
gcc -fopenmp omp_hola.c -o omp_hola
```

To run the code, simply type the command

```
./omp_hola
```

and the program should run.

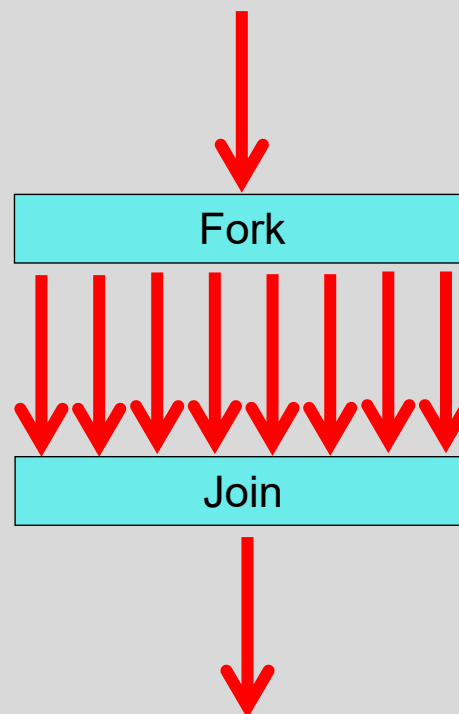
Programming Model – Fork/Join

Serial regions by default, annotate to create *parallel regions*

- Generic parallel regions
- Parallelized loops
- Sectioned parallel regions

Fork /Join model

- Arbitrary number of *logical* thread creation/ destruction events



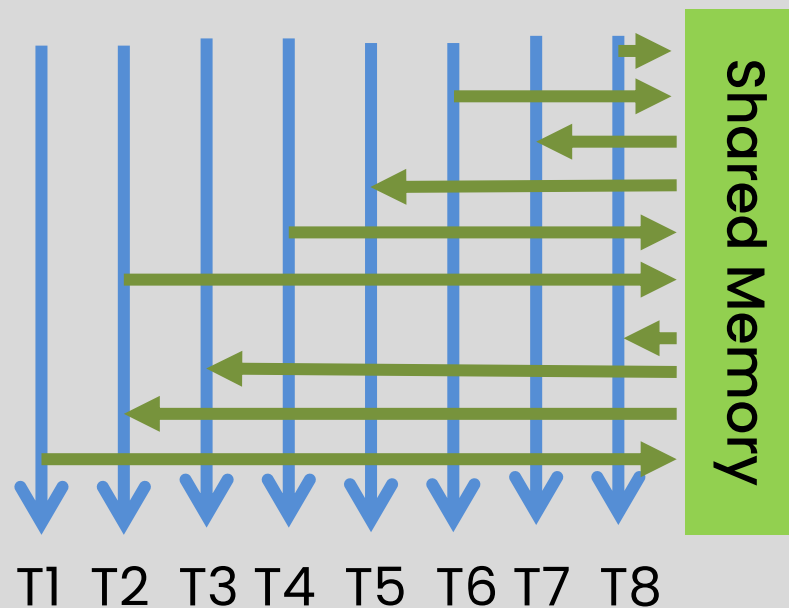
Programming Model – Fork/Join

Serial regions by default, annotate to create *parallel regions*

- Generic parallel regions
- Parallelized loops
- Sectioned parallel regions

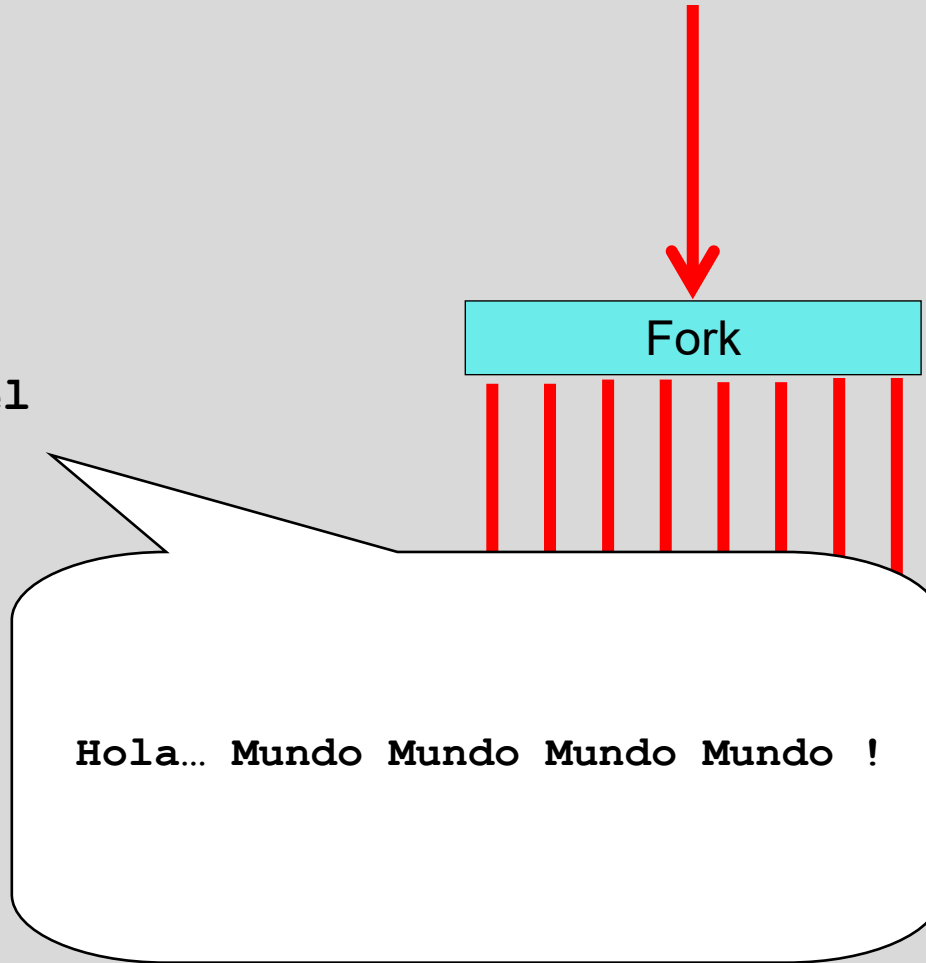
Fork /Join model

- Arbitrary number of *logical* thread creation/ destruction events



Programming Model – Fork/Join

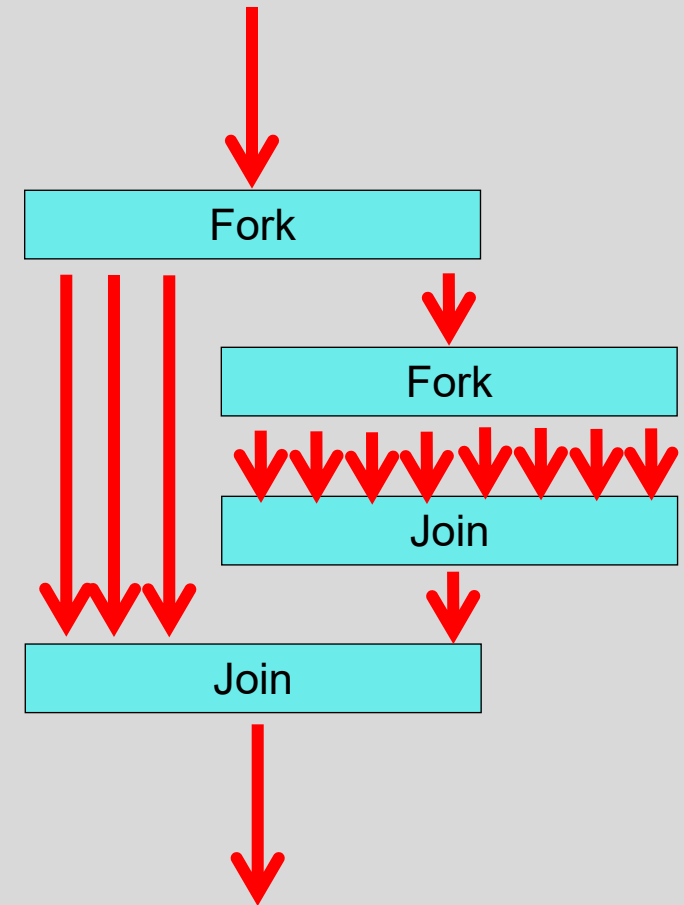
```
int main() {  
    // serial region  
    printf("Hola...");  
    // parallel region  
    #pragma omp parallel  
    {  
        printf("Mundo ");  
    }  
    // serial again  
    printf("!");  
}
```



Programming Model – Nested threads

Fork/Join can be nested

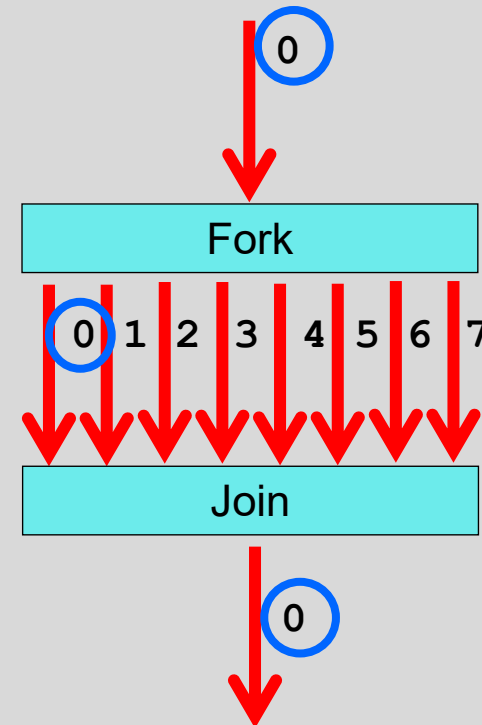
- Nesting complication handled “automagically” at compile-time
- Independent of the number of threads actually running.



Programming Model – Thread Identification

Master Thread

- Thread with ID=0
- Only thread that exists in sequential regions
- Depending on implementation, may have special purpose inside parallel regions
- Some special directives affect only the master thread (like `master`).



Example

```
int main() {  
  
    int tid, nthreads;  
    omp_set_num_threads(16);  
  
    /* Do this part in parallel */  
    #pragma omp parallel private(nthreads, tid)  
    {  
        printf( "Hola Mundo!\n" );  
        tid = omp_get_thread_num();  
        if (tid == 0)  
        {  
            nthreads = omp_get_num_threads();  
            printf("I'm the master, Number of threads = %d\n",  
                nthreads);  
        }  
    }  
  
    return 0;  
}
```

Programming Model – Data/Control Parallelism

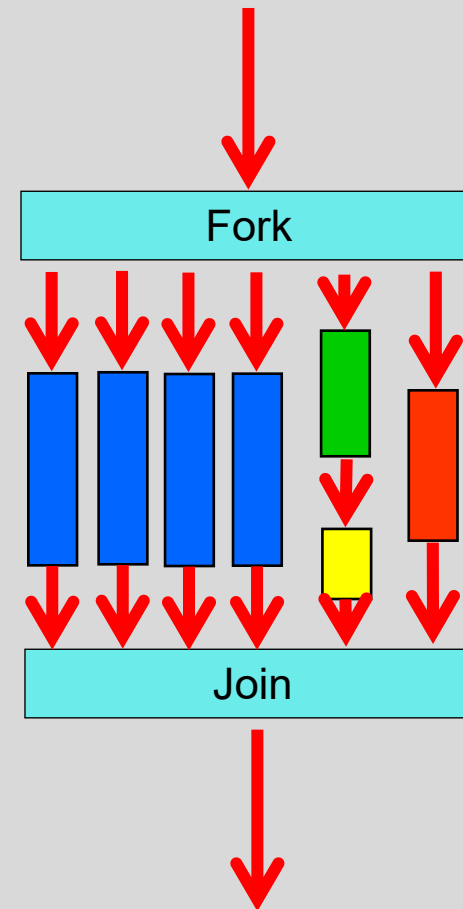
Data parallelism

Threads perform similar functions, guided by thread identifier

Control parallelism

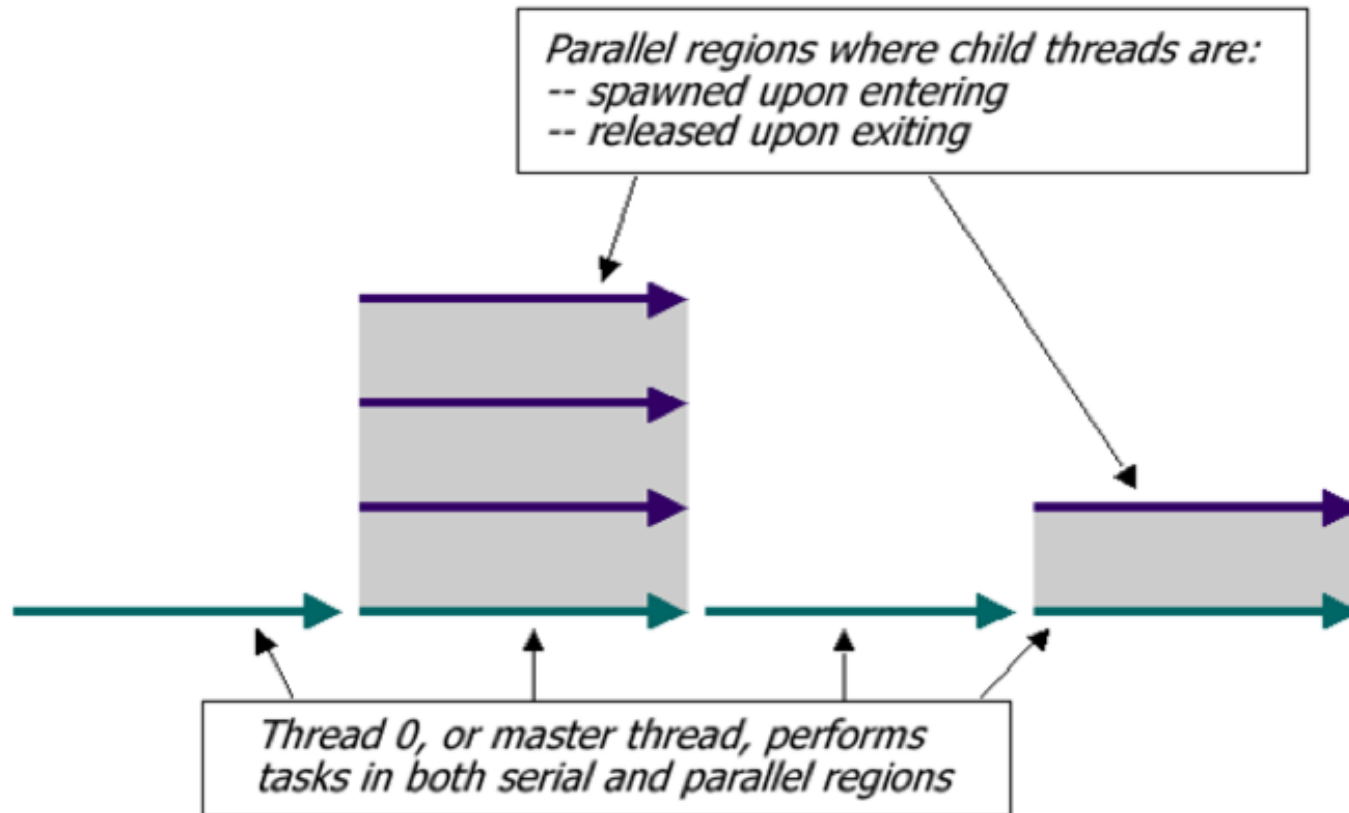
Threads perform differing functions:

- One thread for I/O, one for computation, etc...

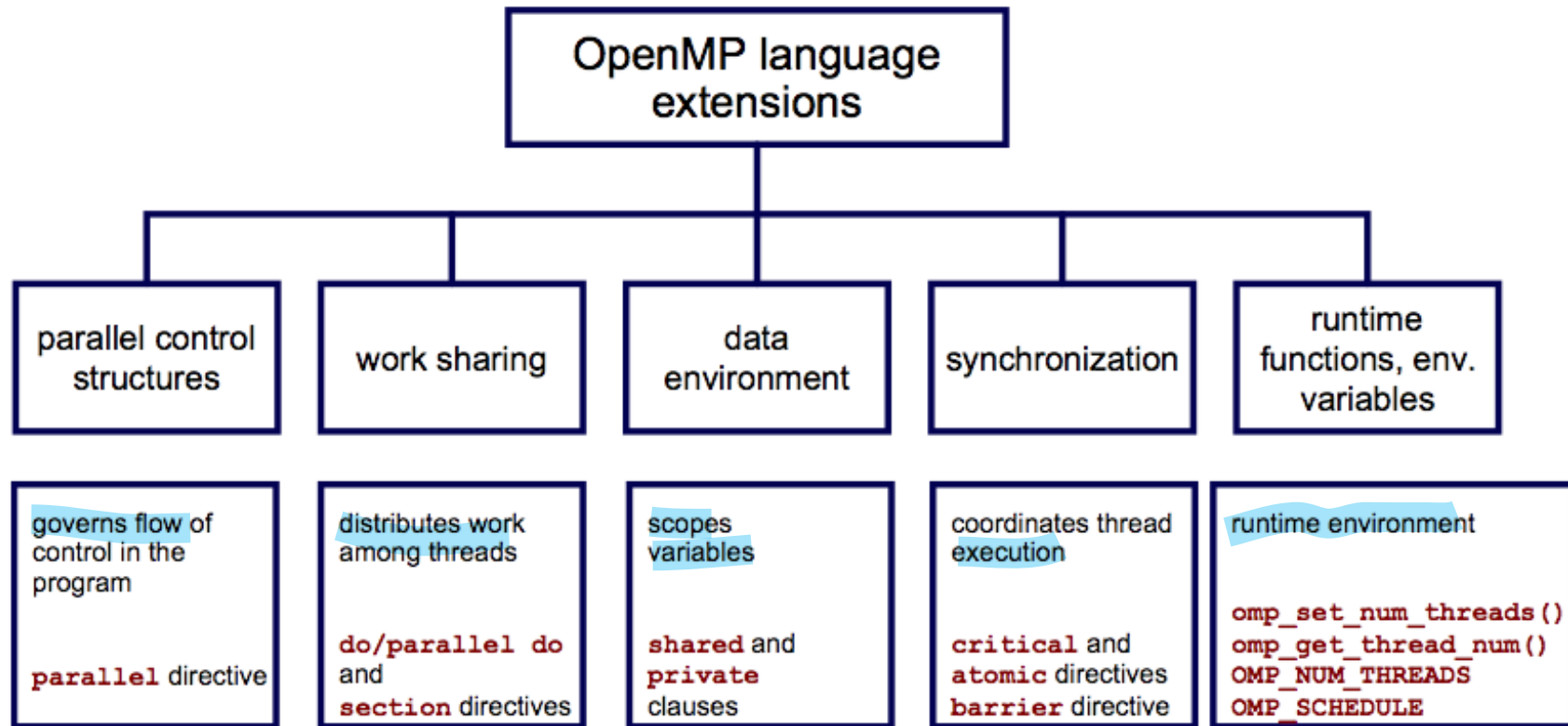


Fork-Join programming model

Fork and Join: Master thread spawns a team of threads as needed



OpenMP Structure



OpenMP functions

Function	Description
<code>void omp_set_num_threads(int)</code>	Sets the number of threads in upcoming parallel regions, unless overridden by a <code>num_threads</code> clause.
<code>int omp_get_num_threads()</code>	Returns the number of threads in the parallel region.
<code>int omp_get_max_threads()</code>	Returns an integer that is equal to or greater than the number of threads that would be available if a parallel region without <code>num_threads</code> were defined at that point in the code.
<code>int omp_get_thread_num()</code>	Returns the thread number of the thread executing within its thread team.
<code>int omp_get_num_procs()</code>	Returns the number of processors that are available when the function is called.
<code>void omp_set_dynamic(int)</code>	Indicates that the number of threads available in upcoming parallel regions can be adjusted by the run time.
<code>int omp_get_dynamic()</code>	Returns a value that indicates if the number of threads available in upcoming parallel regions can be adjusted by the run time.
<code>void omp_set_nested(int)</code>	Enables nested parallelism.
<code>int omp_get_nested()</code>	Returns a value that indicates if nested parallelism is enabled.
<code>double omp_get_wtime()</code>	Returns a value in seconds of the time elapsed from some point.

OpenMP directives

Parallel region construct

```
#pragma omp parallel [clause ...]  
  if (scalar_expression)  
  private (list)  
  shared (list)  
  default (shared | none)  
  firstprivate (list)  
  reduction (operator: list)  
  copyin (list)  
  num_threads (integer-expression)
```

Memory Model

- Shared memory communication
 - Threads cooperates by accessing shared variables
- The sharing is defined syntactically
 - Any variable that is seen by two or more threads is shared
 - Any variable that is seen by one thread only is private
- Race conditions possible
 - Use synchronization to protect from conflicts
 - Change how data is stored to minimize the synchronization

Programming Model – Concurrent Loops

- OpenMP easily parallelizes loops
No data dependencies between iterations!
- Preprocessor calculates loop bounds for each thread directly from *serial* source

```
#pragma omp parallel for
    for( i=0; i < 25; i++ )
    {
        printf("hola mundo\n");
    }
```

The problem

- Executes the same code as many times as there are threads
- How many threads do we have? `omp_set_num_threads(n)`
- What is the use of repeating the same work n times in parallel?
- Can use `omp_thread_num()` to distribute the work between threads.
- **D** is shared between the threads, **i** and **sum** are private

```
double D[1000];
#pragma omp parallel
{
    int i; double sum = 0;
    for (i=0; i<1000; i++) sum += D[i];
    printf("Thread %d computes %f\n", omp_thread_num(), sum);
}
```

Programming Model – Concurrent Loops

Sequential code

```
for (int i=0; i<N; i++) { a[i]=b[i]+c[i]; }
```

(Semi) manual
parallelization

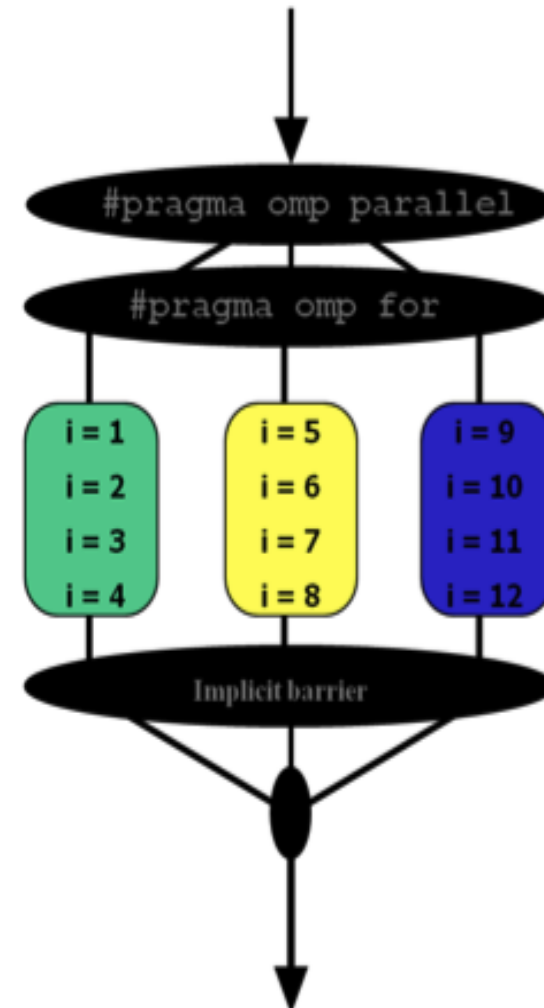
```
#pragma omp parallel  
{  
    int id = omp_get_thread_num();  
    int Nthr = omp_get_num_threads();  
    int istart = id*N/Nthr;  
    int iend = (id+1)*N/Nthr;  
    for (int i=istart; i<iend; i++) {  
        a[i]=b[i]+c[i]; }  
}
```

Automatic
parallelization

```
#pragma omp parallel for schedule(static)  
{  
    for (int i=0; i<N; i++) {  
        a[i]=b[i]+c[i]; }  
}
```

Programming Model – Concurrent Loops

- Used to assign each thread an independent set of iterations
- Threads must wait at the end
- Can combine the directives:
 - `#pragma omp parallel for`
- Only simple kinds of for loops:
 - Only one signed integer variable
 - Initialization: `var=init`
 - Comparison: `var op last`
op: `<`, `>`, `<=`, `>=`
 - Increment: `var++`, `var--`,
`var+=incr`, `var-=incr`, etc.



Programming Model – Concurrent Loops

- Load balancing

If all the iterations execute at the same speed, the processors are used optimally. If some iterations are faster than others, some processors may get idle, reducing the speedup.

We don't always know the distribution of work, may need to re-distribute dynamically.

- Granularity

Thread creation and synchronization takes time assigning work to threads. It may take more time than the execution itself!

It is important to reach a good trade-off between load balancing and granularity!

Controlling Granularity

#pragma omp parallel if (expression)

Can be used to **disable parallelization** in some cases (when the input is determined to be too small to be beneficially multithreaded)

#pragma omp num_threads (expression)

Control the number of threads used for this parallel region

Programming Model – Loop Scheduling

```
#pragma omp for [clause ...]  
    schedule (type [,chunk])  
    ordered  
    private (list)  
    firstprivate (list)  
    lastprivate (list)  
    shared (list)  
    reduction (operator: list)  
    collapse (n)  
    nowait
```

for_loop

Programming Model – Loop Scheduling

- `schedule (type, [chunk])` clause determines how loop iterations are divided among the thread team

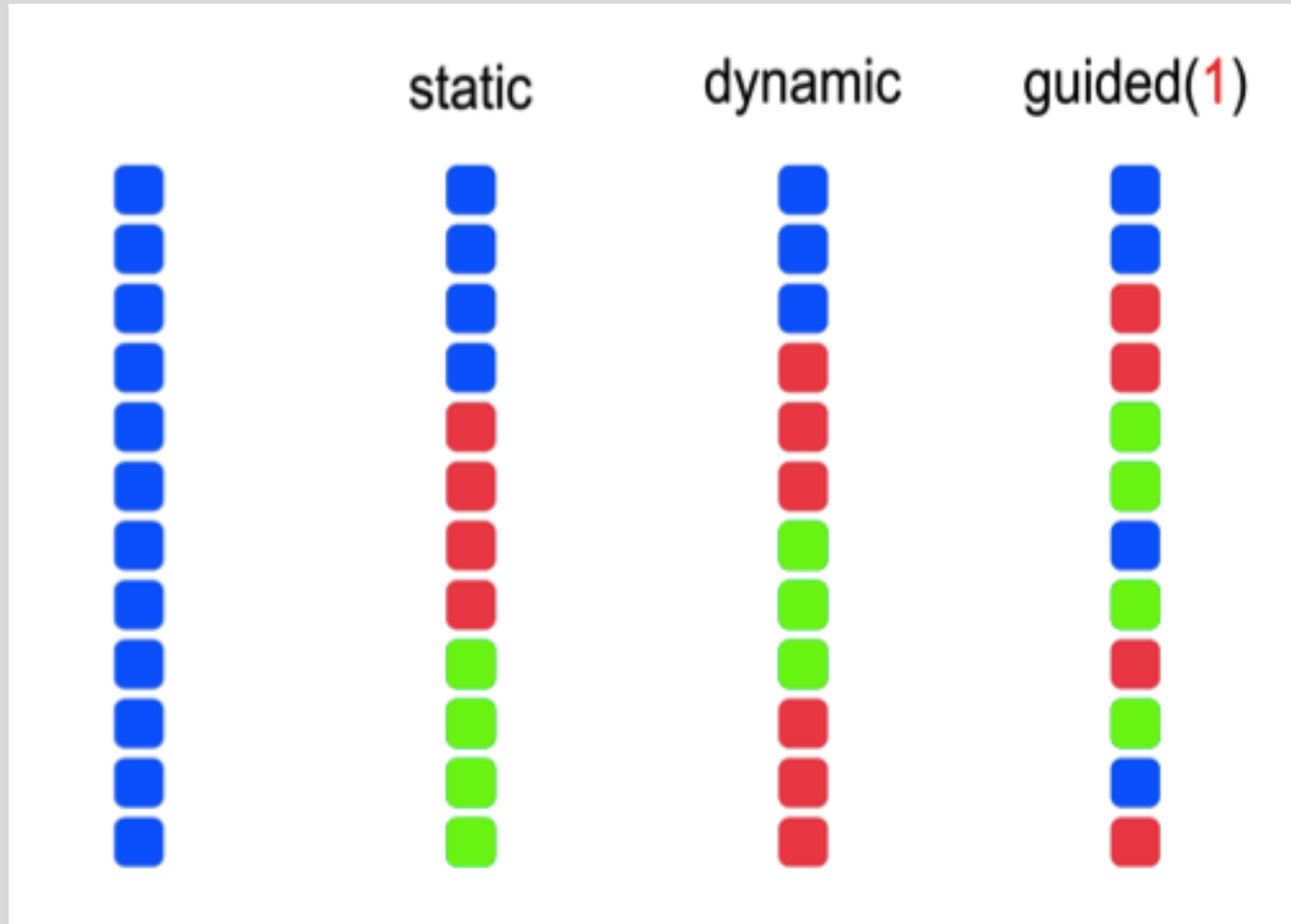
`static ([chunk])` divides iterations statically between threads

Each thread receives `[chunk]` iterations, rounding as necessary to account for all iterations. Default `[chunk]` is $\text{ceil}(\# \text{ iterations} / \# \text{ threads})$

`dynamic ([chunk])` allocates `[chunk]` iterations per thread, allocating an additional `[chunk]` iterations when a thread finishes. Default `[chunk]` is 1

`guided ([chunk])` allocates dynamically, but `[chunk]` is exponentially reduced with each allocation

Programming Model – Loop Scheduling



Data Sharing

- .Shared Memory programming model
 - .Most variables are **shared by default**
 - .We can define a variable **as private**

```
// Do this part in parallel
#pragma omp parallel private(nthreads, tid)
{
    tid = omp_get_thread_num();

    printf( "Hello, World!\n" );
    if (tid == 0)
    {
        ...
    }
}
```

Programming Model – Data Sharing

- Parallel programs often employ two types of data:
 - Shared data, visible to all threads, similarly named
 - Private data, visible to a single thread (often stack-allocated)

```
int bigdata[1024];  
void* foo(void* bar) {  
    int tid;  
    #pragma omp parallel \  
        shared ( bigdata ) \  
        private ( tid )  
    {  
        /* Calc. here */  
    }  
}
```

Programming Model – Data Sharing

private:

A copy of the variable is created for each thread.

No connection between the original variable and the private copies

Can achieve the same using variables inside { }

```
int i;
```

```
#pragma omp parallel for private(i)
```

```
for (i=0; i<n; i++) { ... }
```

Programming Model – Data Sharing

Firstprivate:

Each thread has the same copy of x

Same, but the initial value is copied from the main copy

Lastprivate:

Each thread has private idx, at the end of the loop idx will have the value of the last iteration

Same, but the last value is copied to the main copy

```
int idx=1;
int x = 10;

#pragma omp parallel for \
    firstprivate(x) \
    lastprivate(idx)

for (i=0; i<n; i++) {
    if (data[i]==x) idx = i;
}
```

Threadprivate

Similar to private, but defined per variable
Declaration immediately after variable definition.

Must be visible in all translation units. Persistent between parallel sections

Can be initialized from the master's copy with

#pragma omp copyin

More efficient than private, but a global variable!

```
int data[100];  
#pragma omp threadprivate(data)  every thread has a persistent copy of data  
...  
#pragma omp parallel for copyin(data)  before entering the parallel region the master's data  
for (...)                             is copied to all threads
```

Example

The function TestForPrime (usually) takes little time, but can take long, if the number is a prime indeed.

Solution: use dynamic, but with chunks

Here if we use dynamic, chunks it's faster obviously

```
#pragma omp parallel for schedule ????  
for( int i = start; i <= end; i += 2 )  
{  
    if ( TestForPrime(i) ) gPrimesFound++;  
}
```

Work sharing: Sections

```
answer1 = long_computation_1();  
answer2 = long_computation_2();  
if (answer1 != answer2) { ... }
```

- How to parallelize? These are just two independent computations!

```
#pragma omp sections  
{  
    #pragma omp section  
    answer1 = long_computation_1();  
    #pragma omp section  
    answer2 = long_computation_2();  
}  
if (answer1 != answer2) { ... }
```

Sections

- The **SECTIONS** directive is a non-iterative work-sharing construct. It specifies that the enclosed section(s) of code are to be divided among the threads in the team.
- Independent **SECTION** directives are nested within a SECTIONS directive
- Every **SECTION** is executed once by a thread in the team.
- Different sections may be executed by different threads. It is possible that for a thread to execute more than one section if it is quick enough and the implementation permits such.

Example

```
#include <omp.h>

#define N 1000

main ()
{
    int i;
    float a[N], b[N], c[N], d[N];
    /* Some initializations */
    for (i=0; i < N; i++) {
        a[i] = i * 1.5;
        b[i] = i + 22.35;
    }
```

Example

```
#pragma omp parallel shared(a,b,c,d) private(i)
{
    #pragma omp sections
    {
        #pragma omp section
        for (i=0; i < N; i++)
            c[i] = a[i] + b[i];

        #pragma omp section
        for (i=0; i < N; i++)
            d[i] = a[i] * b[i];
    } /* end of sections */
} /* end of parallel section */
}
```

Synchronization

What should the result be (assuming 2 threads)?

```
X=0;
```

```
#pragma omp parallel
```

```
X = X+1;
```

Synchronization

- **2**

is the expected answer, but can be 1 with unfortunate interleaving

- OpenMP assumes that the programmer knows what she/he is doing.
- Regions of code that are marked to run in parallel are independent. If access collisions are possible, it is the programmer's responsibility to insert protection.

Synchronization

OpenMP Synchronization

- `Nowait` (turn synchronization off!)
- `Single/Master` execution
- `Critical sections`, `Atomic updates`
- `Ordered`
- `Barriers`
- `Flush` (memory subsystem synchronization)
- `Reduction` (special case)

Single/Master

```
#pragma omp single
```

- Only one of the threads will execute the following block of code
- The rest will wait for it to complete
- Good for non-thread-safe regions of code (such as I/O)
- Must be used in a parallel region
- Applicable to parallel for sections

Single/Master

#pragma omp master

The following block will be executed by the master thread

No synchronization involved

Applicable only to parallel sections

```
#pragma omp parallel
{
do_preprocessing () ;
#pragma omp single
read_input () ;
#pragma omp master
notify_input_consumed () ;
do_processing () ; }
```

Critical Sections

```
#pragma omp critical [name]
```

Standard critical section functionality

- Critical sections are global in the program
Can be used to protect a single resource in different functions
- Critical sections are identified by the name
All the unnamed critical sections are mutually exclusive throughout the program
- All the critical sections having the same name are mutually exclusive between themselves.

Critical Sections

```
int x=0;  
#pragma omp parallel shared(x)  
{  
#pragma omp critical  
    x++;  
}
```

Ordered

`#pragma omp ordered statement`

Executes the statement in the sequential order of iterations

Example:

```
#pragma omp parallel for ordered
for (j=0; j<N; j++) {
    int result = j*j;
    #pragma omp ordered
    printf ("computation(%d) = %d\n" ,j , result ) ;
}
```

Barrier synchronization

`#pragma omp barrier`

Performs a **barrier synchronization** between all the threads in a team at the given point.

Example:

```
#pragma omp parallel
{
    int result = heavy_computation_part1 ();
    #pragma omp atomic
    sum += result ;
    #pragma omp barrier
    heavy_computation_part2 (sum) ;
}
```

Explicit Locking

Can be used to pass lock variables around (unlike critical sections!)

Can be used to implement more involved synchronization constructs

Functions:

```
omp_init_lock(), omp_destroy_lock(),  
omp_set_lock(), omp_unset_lock(),  
omp_test_lock()
```

Use `#pragma omp flush` to synchronize memory

Reduction

How to parallelize this code?

sum is not private, but accessing it atomically is too expensive

```
for (j=0; j<N; j++) {  
    sum = sum + a[j]*b[j];  
}
```

Have a private copy of sum in each thread, then add them up

Use the **reduction** clause!

```
#pragma omp parallel for reduction(+: sum)
```

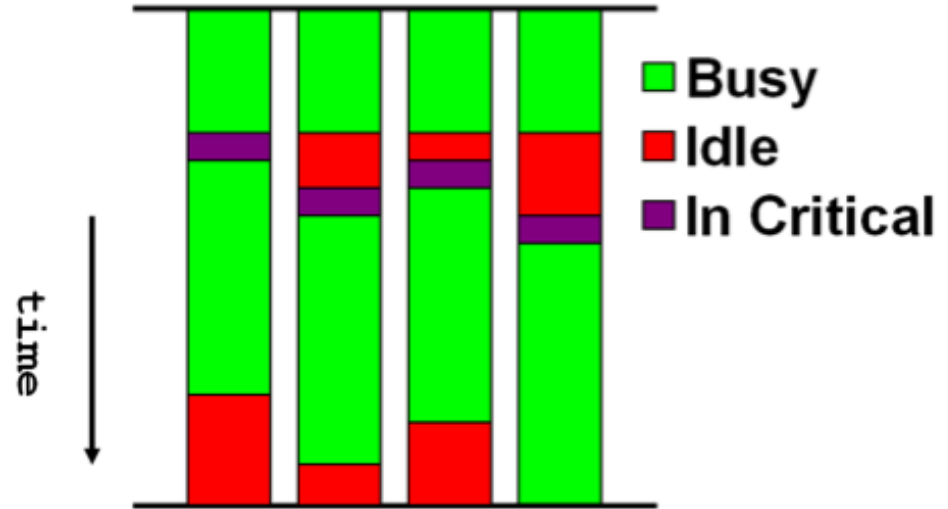
An operator could be used: +, -, *...

Synchronization Overhead

Lost time waiting for locks

Prefer to use structures that are as lock-free as possible!

```
#pragma omp parallel  
{  
    #pragma omp critical  
    {  
        ...  
    }  
    ...  
}
```



Summary

OpenMP

- it is a compiler-based technique to create concurrent code from (mostly) serial code
- can enable (easy) parallelization of loop-based code
- Lightweight syntactic language extensions
- Scalable
- Portable

Not a silver bullet for all applications

More Information

- www.openmp.org

- OpenMP official site

- www.llnl.gov/computing/tutorials/openMP/

- A handy OpenMP tutorial

- www.nersc.gov/nusers/help/tutorials/openmp/

- Another OpenMP tutorial and reference